

Guia de Estudio Go para Backend HTTP

Objetivo

Aprender de forma practica los conceptos clave de Go para desarrollo backend:

- Handler
- Middleware
- Context
- ServeMux
- Struct
- Pointer (*)
- Interfaces y composition
- Profiling con pprof

Esta guia esta pensada para una persona sin experiencia previa en Go.

1. Handler

Definicion

Un handler es una funcion o metodo que responde una ruta HTTP.

Firma comun en Go:

```
func(w http.ResponseWriter, r *http.Request)
```

Que aprender

1. Leer path params y query params.
2. Parsear body JSON.
3. Responder con status code, headers y JSON.

Ejercicio

1. Crear endpoint GET /ping que responda {"message": "pong"}.
 2. Agregar test con httpptest.
-

2. Middleware

Definicion

Capa intermedia que envuelve handlers para ejecutar logica transversal.

Ejemplos:

- Logging
- Request ID

- Recover panic
- Auth

Patron en Go

```
type Middleware func(http.Handler) http.Handler
```

Que aprender

1. Orden de ejecucion de middlewares.
2. Encadenado (chain) de middlewares.
3. Separacion de responsabilidades.

Ejercicio

1. Crear middleware `AddPoweredBy` con header `X-Powered-By: Go`.
 2. Validar en test que siempre se agrega.
-

3. Context

Definicion

Objeto para cancelacion, timeouts y metadatos por request.

Casos de uso

- Cortar llamadas externas cuando vence timeout.
- Propagar `request_id`.
- Cancelar procesos cuando el cliente cierra conexion.

Que aprender

1. `r.Context()` en handlers.
2. `context.WithTimeout`.
3. `context.WithValue` con keys tipadas.

Ejercicio

1. Agregar timeout de 2s en una llamada externa.
 2. Verificar manejo de error por deadline.
-

4. ServeMux

Definicion

Router HTTP estandar de Go (`net/http`).

Que aprender

1. Registro de rutas.
2. Patrones por metodo ("GET /ruta" en Go moderno).
3. Rutas con parametros.

Ejercicio

1. Registrar GET /api/v1/version.
 2. Probar request valida e invalida.
-

5. Struct

Definicion

Tipo compuesto para agrupar datos relacionados.

Ejemplo:

```
type Pokemon struct {  
    ID    int    `json:"id"`  
    Name  string `json:"name"`  
}
```

Que aprender

1. Modelar request/response.
2. Usar tags JSON correctamente.
3. Diferenciar modelos internos y externos.

Ejercicio

1. Crear struct de error API con `code`, `message`, `request_id`.
 2. Responder errores usando esa estructura.
-

6. Pointer (*)

Definicion

Referencia a un valor. Sirve para evitar copias y permitir mutaciones compartidas.

Comparacion:

- `cfg config.Config` -> copia del struct.
- `cfg *config.Config` -> referencia al original.

Que aprender

1. Cuándo usar receiver por valor y por puntero.
2. Mutabilidad compartida.
3. Coste de copia de structs grandes.

Ejercicio

1. Probar metodo con receiver valor y puntero.
 2. Ver como cambia el comportamiento al mutar campos.
-

7. Interfaces y Composition

Interfaces

Contratos de comportamiento. Se satisfacen implícitamente en Go.

Composition

Combinar objetos pequeños en vez de herencia.

Que aprender

1. Interfaces pequeñas.
2. Inyeccion de dependencias.
3. Testear con mocks/fakes.

Ejercicio

1. Definir interface `PokemonService`.
 2. Hacer que el handler dependa de la interface y no de implementacion concreta.
 3. Mockear esa interface en tests.
-

8. Profiling con pprof

Objetivo

Medir rendimiento real de CPU y memoria.

Que aprender

1. Exponer pprof en dev.
2. Capturar perfil CPU.
3. Capturar perfil heap.
4. Interpretar resultados y optimizar.

Comandos

```
go tool pprof http://localhost:8080/debug/pprof/profile?seconds=30
go tool pprof http://localhost:8080/debug/pprof/heap
```

Ejercicio

1. Medir endpoint bajo carga.
 2. Detectar funcion lenta.
 3. Optimizar.
 4. Volver a medir.
-

Plan de estudio (10 dias)

1. Dia 1: handlers basicos y respuestas JSON.
 2. Dia 2: ServeMux y enrutamiento.
 3. Dia 3: structs y tags JSON.
 4. Dia 4: pointers y receivers.
 5. Dia 5: middleware chain.
 6. Dia 6: context y timeouts.
 7. Dia 7: interfaces pequeñas.
 8. Dia 8: composition e inyeccion.
 9. Dia 9: tests con `httptest`.
 10. Dia 10: pprof + mejora medida.
-

Checklist final

1. Crear endpoint nuevo con test.
 2. Explicar flujo middleware -> handler.
 3. Distinguir valor vs puntero.
 4. Aplicar interface para desacoplar.
 5. Propagar context correctamente.
 6. Sacar perfil pprof y detectar cuello de botella.
-

Recomendacion practica

Aprende con cambios pequeños y frecuentes:

- Un cambio funcional.
- Un test.
- Medicion (si aplica).
- Commit.

Ese ciclo corto te da velocidad y seguridad en Go.